

Deep Research: Avatrada Ui 9 Topic 002

Engineering Report: Multi-Monitor Electron Architecture for a Trading Terminal

TO: Lead Architect **FROM:** Autonomous Technical Researcher **DATE:** October 26, 2023 **SUBJECT:** Deep-Dive on Multi-Process Orchestration, State Synchronization, and Resilience for the Avatrada Trading Terminal

1.0 Executive Summary

This report provides a detailed engineering analysis of the core architectural components required to build a resilient, multi-monitor trading terminal using the Electron framework. The system is designed around three distinct renderer processes (windows) that must function as a cohesive application. This document deconstructs the necessary Inter-Process Communication (IPC) patterns, state management strategies, window orchestration logic, and crash recovery mechanisms.

The proposed architecture establishes the **Main Process as the central authority or "Shared Brain"** for application state, while leveraging the `BroadcastChannel` API for low-latency, peer-to-peer UI coordination. This hybrid approach ensures data integrity and consistency for critical state (e.g., account balances, positions) via the Main Process, while enabling highly responsive UI interactions (e.g., theme changes, focus tracking) that bypass the main thread bottleneck. The report provides production-ready code patterns, implementation strategies, and a critical analysis of potential failure modes and optimizations for each component.

2.0 IPC Architecture for Redux State Synchronization

2.1 Technical Deconstruction

In an Electron application with multiple `BrowserWindow` instances, each renderer process is an isolated environment with its own memory and state. To synchronize a Redux store across these processes, we must treat the Main Process as the single source of truth.

The pattern, often called the "Shared Brain" or "Main Process as a Service," works as follows:

1. **Centralized Store:** A single Redux store instance lives in the Main Process. This store holds the global application state.
2. **Action Dispatch Proxy:** When a renderer process (e.g., "Order Entry" window) wants to change the state, it does not update its local store directly. Instead, it dispatches the action to the Main Process via IPC.
3. **State Update & Broadcast:** The Main Process receives the action, applies it to its central store, and calculates the new state. It then broadcasts this new, authoritative state to *all* renderer processes.
4. **State Rehydration:** Each renderer receives the new state from the Main Process and updates its local Redux store, causing its React components to re-render. This ensures all windows are perfectly synchronized.

This architecture prevents race conditions and state divergence that would occur if renderers updated their state independently.

2.2 Implementation Strategy

This requires a custom Redux middleware in the renderer processes and a set of IPC listeners in the Main Process.

Main Process (`main.js`) Relay Logic

```
// main.js
import { app, BrowserWindow, ipcMain } from 'electron';
import { createStore } from 'redux';
import rootReducer from './reducers'; // Your combined Redux reducers

// 1. Create the single source of truth store in the Main process
const centralStore = createStore(rootReducer);
let windows = []; // Keep track of all windows

// 2. Listener for actions dispatched from any renderer
ipcMain.on('redux-action', (event, action) => {
  // Dispatch the action to the central store
  centralStore.dispatch(action);

  // Get the updated state
  const updatedState = centralStore.getState();

  // 3. Broadcast the new state to all windows
  windows.forEach(win => {
    if (win && !win.isDestroyed()) {
      win.webContents.send('redux-state-update', updatedState);
    }
  });
});

// 4. Listener to provide initial state to a newly opened window
ipcMain.handle('redux-get-initial-state', () => {
  return centralStore.getState();
});

function createWindow() {
  // ... create BrowserWindow instances
  const win = new BrowserWindow({ /* ... webPreferences ... */ });
  windows.push(win);
  win.on('closed', () => {
    windows = windows.filter(w => w !== win);
  });
}
```

```
app.whenReady().then(createWindow);
```

Renderer Process (preload.js and Redux Middleware)

First, expose the IPC channels securely using the `contextBridge`.

```
// preload.js
const { contextBridge, ipcRenderer } from 'electron';

contextBridge.exposeInMainWorld('electronRedux', {
  sendAction: (action) => ipcRenderer.send('redux-action', action),
  getInitialState: () => ipcRenderer.invoke('redux-get-initial-state'),
  onStateUpdate: (callback) => ipcRenderer.on('redux-state-update', (event,
    state) => callback(state)),
});
```

Next, create the Redux middleware and store configuration.

```
// renderer/store.js
import { createStore, applyMiddleware } from 'redux';
import rootReducer from './reducers';

// The middleware intercepts actions before they hit the local reducer
const ipcRelayMiddleware = store => next => action => {
  // Only forward standard actions, not internal/hydration ones
  if (!action.meta || !action.meta.fromMain) {
    window.electronRedux.sendAction(action);
  }
  // We don't call `next(action)` here because the action will be processed
  // by the main process. The renderer will receive a full state update.
  // This prevents the action from being applied twice.
  return;
};

// A special reducer to handle the full state hydration from the main process
const rehydrationReducer = (state, action) => {
  if (action.type === 'STATE_HYDRATE') {
```

```

    return action.payload;
  }
  return rootReducer(state, action);
};

export async function configureStore() {
  const initialState = await window.electronRedux.getInitialState();
  const store = createStore(rehydrationReducer, initialState,
    applyMiddleware(ipcRelayMiddleware));

  // Listen for state updates from the main process
  window.electronRedux.onStateUpdate((newState) => {
    store.dispatch({
      type: 'STATE_HYDRATE',
      payload: newState,
      meta: { fromMain: true } // Meta flag to avoid re-broadcasting
    });
  });

  return store;
}

```

2.3 Critical Analysis

- **Failure Modes:** The Main Process is a single point of failure. If it crashes, the entire application terminates. The IPC channel is generally reliable, but extreme load could theoretically introduce latency, although this is rare on a local machine.
 - **Edge Cases:** High-frequency actions (e.g., from rapid mouse movements) could flood the IPC channel. Actions should be debounced or throttled where appropriate before being dispatched.
 - **Optimizations:** For very large state objects, broadcasting the entire state on every action can be inefficient. An optimization is to compute a state diff (e.g., using libraries like `fast-json-patch`) in the Main Process and broadcast only the patch. The renderers would then apply the patch to their local state. This significantly reduces the data payload over IPC.
-

3.0 Programmatic Multi-Monitor Window Positioning

3.1 Technical Deconstruction

The `screen` module in Electron provides the necessary tools to identify and inspect all connected displays. The `screen.getAllDisplays()` method returns an array of `Display` objects, each containing properties like `id`, `bounds` (x, y, width, height), `size` (pixel dimensions), and `scaleFactor`.

The strategy is to: 1. Fetch all connected displays. 2. Verify that the required number of displays (three) are present. 3. Sort the displays geographically (by their `bounds.x` coordinate) to reliably identify left, center, and right. 4. Identify the specific monitors by their resolution (1920x1080 for left/right, 2560x1440 for center). 5. Instantiate each `BrowserWindow` with the `x` and `y` coordinates from the corresponding display's `bounds` object.

3.2 Implementation Strategy

This function should be called from the Main Process after the `app` is `ready`.

```
// main.js
import { screen, BrowserWindow } from 'electron';

function positionWindowsOnMonitors() {
  const displays = screen.getAllDisplays();

  // 1. Verify we have at least 3 monitors
  if (displays.length < 3) {
    console.error("Required 3-monitor setup not detected. Found:",
displays.length);
    // Fallback to default window creation
    createDefaultWindows();
    return;
  }

  // 2. Sort displays from left to right based on their x-coordinate
```

```
const sortedDisplays = displays.sort((a, b) => a.bounds.x - b.bounds.x);

// 3. Identify the specific monitors based on resolution
const leftMonitor = sortedDisplays[0]; // Assume first is left
const centerMonitor = sortedDisplays[1]; // Assume second is center
const rightMonitor = sortedDisplays[2]; // Assume third is right

// Sanity check the resolutions
const isSetupCorrect =
  leftMonitor.size.width === 1920 && leftMonitor.size.height === 1080 &&
  centerMonitor.size.width === 2560 && centerMonitor.size.height === 1440 &&
  rightMonitor.size.width === 1920 && rightMonitor.size.height === 1080;

if (!isSetupCorrect) {
  console.warn("Monitor resolutions do not match the expected
1080p-1440p-1080p setup. Positioning may be incorrect.");
}

// 4. Create and position the windows
const windowA = new BrowserWindow({
  x: leftMonitor.bounds.x,
  y: leftMonitor.bounds.y,
  width: leftMonitor.bounds.width,
  height: leftMonitor.bounds.height,
  // ... other options
});

const windowB = new BrowserWindow({
  x: centerMonitor.bounds.x,
  y: centerMonitor.bounds.y,
  width: centerMonitor.bounds.width,
  height: centerMonitor.bounds.height,
  // ... other options
});

const windowC = new BrowserWindow({
  x: rightMonitor.bounds.x,
  y: rightMonitor.bounds.y,
  width: rightMonitor.bounds.width,
  height: rightMonitor.bounds.height,
```

```

    // ... other options
  });

  // Load content into windows
  // windowA.loadFile(...) etc.
}

app.whenReady().then(positionWindowsOnMonitors);

```

3.3 Critical Analysis

- **Failure Modes:** If the user disconnects a monitor while the application is running, the windows will remain but the layout context is lost. The application should listen to the `screen.on('display-removed')` and `screen.on('display-added')` events to handle such changes dynamically, perhaps by consolidating windows or alerting the user.
 - **Edge Cases:**
 - **Display Scaling (DPI):** The `bounds` property is in physical pixels. `BrowserWindow` coordinates are expected in DIPs (Device-Independent Pixels). On high-DPI displays, `scaleFactor` will be > 1 . For correct positioning, coordinates must be divided by the scale factor: `x: display.bounds.x / display.scaleFactor`. The provided code assumes a scale factor of 1 for simplicity but should be adjusted for production.
 - **Vertical Monitor Layout:** The `sort((a, b) => a.bounds.x - b.bounds.x)` logic is robust for any horizontal arrangement but may not be sufficient for complex vertical or T-shaped layouts. The logic may need to be adapted if such layouts are to be supported.
 - **Optimizations:** For persistence, after the first successful layout, store the `id` of each display in a configuration file (`electron-store` is excellent for this). On subsequent launches, attempt to find displays by their stored `id` first. This is more resilient to changes in port connections or OS-level display ordering than relying solely on resolution and position.
-

4.0 Low-Latency UI Sync with BroadcastChannel API

4.1 Technical Deconstruction

The `BroadcastChannel` API is a standard web technology that allows scripts from the same origin to send and receive messages. In Electron, since all `BrowserWindow` instances load content from the same origin (e.g., `file://`), they can communicate directly with each other, peer-to-peer, without involving the Main Process.

This is ideal for non-critical, high-frequency UI events where the sub-5ms latency is paramount:

- * Synchronizing hover effects across windows.
- * Toggling a theme (dark/light mode) instantly everywhere.
- * Notifying other windows which component has focus.

The mechanism is simple: create a `BroadcastChannel` with a shared name in each renderer. One window calls `postMessage()`, and all other windows receive the message via an `onmessage` event handler.

4.2 Implementation Strategy

This code is executed entirely within the renderer processes.

```
// In any renderer process (e.g., Window A)

// 1. Create a channel. The name 'ui-coordination' must be the same in all
// windows.
const uiChannel = new BroadcastChannel('ui-coordination');

// Function to send a message
function toggleTheme(theme) {
  console.log('Broadcasting theme change:', theme);
  uiChannel.postMessage({
    type: 'THEME_CHANGE',
    payload: { theme: theme }
  });
}
```

```

}

// Example usage:
// document.getElementById('theme-button').onclick = () => toggleTheme('dark');

```

```

// In other renderer processes (e.g., Window B and C)

// 1. Create a channel with the same name to listen for messages.
const uiChannel = new BroadcastChannel('ui-coordination');

// 2. Set up a listener to handle incoming messages.
uiChannel.onmessage = (event) => {
  const { type, payload } = event.data;

  switch (type) {
    case 'THEME_CHANGE':
      console.log('Theme change received:', payload.theme);
      // Apply the theme change to the document body or root component
      document.body.className = payload.theme;
      break;
    case 'FOCUS_ELEMENT':
      // Logic to highlight a corresponding element
      break;
    default:
      console.warn('Unknown message type received:', type);
  }
};

// It's good practice to close the channel when the window unloads
window.addEventListener('beforeunload', () => {
  uiChannel.close();
});

```

4.3 Critical Analysis

- **Failure Modes:** `BroadcastChannel` is "fire-and-forget." There is no acknowledgment of receipt. If a receiving window is in the middle of a long-running task and its event loop is blocked, the message may be delayed or

dropped. Therefore, it is **unsuitable for critical state changes** that must be guaranteed.

- **Edge Cases:** The data sent via `postMessage` must be serializable using the structured clone algorithm. This means you cannot send functions, DOM nodes, or other complex objects.
 - **Optimizations:** Use distinct channel names for different domains of communication (e.g., `new BroadcastChannel('theme-sync')`, `new BroadcastChannel('focus-sync')`). This prevents a single `onmessage` handler from becoming a complex switch statement and improves separation of concerns.
-

5.0 Graceful Degradation and Crash Recovery

5.1 Technical Deconstruction

Electron's multi-process architecture means a renderer process can crash without taking down the Main Process or other renderers. The `webContents` object associated with each `BrowserWindow` emits a `crashed` event when this happens.

The strategy for graceful degradation is: 1. **Detection:** The Main Process listens for the `crashed` event on the `webContents` of each critical window (e.g., the "Market Data" window). 2. **Notification:** Upon detecting a crash, the Main Process sends a specific IPC message to all other surviving windows. 3. **Alerting:** The surviving renderer processes listen for this IPC message and react by displaying a non-intrusive notification (e.g., a toast or banner) to the user, informing them that a part of the application is unavailable. 4. **State Management:** The UI in the surviving windows can be updated to disable features that depended on the crashed window.

5.2 Implementation Strategy

Main Process: Crash Detection and Notification

```
// main.js
import { BrowserWindow, ipcMain } from 'electron';

// Store windows in an object for easy identification
const appWindows = {
  marketData: null,
  orderEntry: null,
  charts: null,
};

function createAllWindows() {
  // ... create orderEntry and charts windows

  appWindows.marketData = new BrowserWindow({ /* ... */ });
  appWindows.marketData.loadFile('market-data.html');

  // 1. Attach a crash listener
  appWindows.marketData.webContents.on('crashed', (event, killed) => {
    console.error(`Market Data window crashed! Was killed: ${killed}`);

    // 2. Notify all other living windows
    const survivingWindows = [appWindows.orderEntry, appWindows.charts];
    survivingWindows.forEach(win => {
      if (win && !win.isDestroyed()) {
        win.webContents.send('child-window-crashed', {
          name: 'Market Data',
          crashedAt: new Date().toISOString()
        });
      }
    });

    // Optional: Attempt to restart the window after a delay
    // setTimeout(recreateMarketDataWindow, 5000);
  });
}
```

```
appWindows.marketData.on('closed', () => {
  appWindows.marketData = null;
});
}
```

Renderer Process: Receiving Alert and Updating UI

First, expose the listener in `preload.js`.

```
// preload.js
contextBridge.exposeInMainWorld('appEvents', {
  onWindowCrash: (callback) => ipcRenderer.on('child-window-crashed', (event,
data) => callback(data)),
});
```

Then, use it in the renderer's UI logic (e.g., a top-level React component).

```
// renderer/App.js (React example)
import React, { useEffect, useState } from 'react';

function App() {
  const [crashInfo, setCrashInfo] = useState(null);

  useEffect(() => {
    window.appEvents.onWindowCrash((data) => {
      console.warn(`${data.name} window has crashed.`);
      setCrashInfo(data);
      // Optionally, set a timer to hide the notification after a while
      setTimeout(() => setCrashInfo(null), 15000);
    });
  }, []);

  return (
    <div>
      {crashInfo && (
        <div className="crash-notification-banner">
          Warning: The '{crashInfo.name}' window has become unresponsive and
closed. Some functionality may be unavailable.
        </div>
      )}
    </div>
  );
}
```

```
        </div>
    })
    { /* Rest of the application UI */ }
</div>
);
}
```

5.3 Critical Analysis

- **Failure Modes:** If the Main Process crashes, this entire system fails. Robust logging (e.g., to a file) within the crash handler is essential for debugging production issues.
- **Edge Cases:** A window might become unresponsive (`unresponsive` event) before it fully crashes. It's often better to listen for both events. A "flapping" window that repeatedly crashes and restarts can degrade system performance. The restart logic should include an exponential backoff strategy to prevent this.
- **Optimizations:** The user notification could be more interactive, offering a "Restart Window" button. Clicking this button would send an IPC message back to the Main Process, instructing it to re-create the specific `BrowserWindow`. When the window is recreated, it must re-request the initial state from the Main Process to ensure it is in sync (linking back to the pattern in Section 2.0).